

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Chí Công - Nguyễn Hưng

XÂY DỰNG HỆ THỐNG KIỂM THỬ TỰ
ĐỘNG
ỨNG DỤNG WEB

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

Tp. Hồ Chí Minh, tháng MM/YYYY

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Chí Công - 22120038

Nguyễn Hưng - 22120122

XÂY DỰNG HỆ THỐNG KIỂM THỬ TỰ
ĐỘNG
ỨNG DỤNG WEB

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

GIẢNG VIÊN HƯỚNG DẪN

PGS.TS. Nguyễn Văn Vũ

Tp. Hồ Chí Minh, tháng MM/YYYY

Lời cam đoan

Chúng tôi xin cam đoan rằng khóa luận tốt nghiệp “Xây dựng hệ thống kiểm thử tự động ứng dụng Web” là công trình nghiên cứu độc lập của chúng tôi, được thực hiện dưới sự hướng dẫn khoa học của PGS.TS. Nguyễn Văn Vũ, Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh.

Toàn bộ nội dung trình bày trong khóa luận này, bao gồm các phân tích, thiết kế, cài đặt và kết quả thực nghiệm, đều do chúng tôi trực tiếp thực hiện và chưa từng được công bố trong bất kỳ công trình nào khác. Các tài liệu, số liệu và kết quả trích dẫn từ nguồn bên ngoài đều được chú thích rõ ràng theo quy định. Chúng tôi hoàn toàn chịu trách nhiệm về tính trung thực và chính xác của những nội dung được trình bày.

Thành phố Hồ Chí Minh, tháng 6 năm 2026

Nguyễn Chí Công

Nguyễn Hưng

Thuyết minh chỉnh sửa đề tài

Bản thuyết minh chỉnh sửa đề tài (nếu có thay đổi tên đề tài, nội dung báo cáo, ... trong cuốn báo cáo sau bảo vệ)

Nhận xét hướng dẫn

Bản nhận xét của giảng viên hướng dẫn (có chữ ký) do giáo vụ cung cấp.

Nhận xét phản biện

Bản nhận xét của giảng viên phản biện (có chữ ký) do giáo vụ cung cấp.

Lời cảm ơn

Để hoàn thành khóa luận này, chúng tôi đã nhận được sự hỗ trợ và đồng hành của nhiều cá nhân và tập thể mà chúng tôi trân trọng ghi nhận.

Trước hết, chúng tôi xin bày tỏ lòng biết ơn sâu sắc đến PGS.TS. Nguyễn Văn Vũ, người đã tận tình định hướng nghiên cứu, góp ý về phương pháp tiếp cận và dành thời gian đọc, nhận xét qua từng giai đoạn của quá trình thực hiện. Những phản hồi khoa học và kinh nghiệm thực tiễn của thầy là nền tảng quan trọng giúp chúng tôi hình thành tư duy hệ thống và giải quyết các vấn đề kỹ thuật phát sinh trong đề tài.

Chúng tôi cũng xin gửi lời cảm ơn chân thành đến quý thầy cô Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên, đã trang bị cho chúng tôi nền tảng kiến thức và kỹ năng trong suốt bốn năm học. Môi trường học thuật nghiêm túc và cởi mở của Khoa chính là điều kiện để chúng tôi có thể theo đuổi và hoàn thiện đề tài này.

Lời cảm ơn đặc biệt xin được gửi đến gia đình, những người luôn là chỗ dựa tinh thần vững chắc trong những thời điểm khó khăn nhất của quá trình nghiên cứu. Sự tin tưởng và khích lệ của gia đình là động lực lớn để chúng tôi kiên trì đến cuối.

Mặc dù đã cố gắng hết sức, khóa luận chắc chắn còn những điểm chưa hoàn thiện. Chúng tôi rất mong nhận được sự góp ý từ hội đồng phản biện và các bạn đọc để tiếp tục cải thiện trong những công trình tiếp theo.

Thành phố Hồ Chí Minh, tháng 6 năm 2026

Nguyễn Chí Công

Nguyễn Hưng

Đề cương chi tiết

Bản đề cương đã thực hiện và nộp cho giáo vụ trước đó (*phải có chữ ký của giảng viên hướng dẫn*).

Mục lục

Thuyết minh chỉnh sửa đề tài	ii
Nhận xét của GV hướng dẫn	iii
Nhận xét của GV phản biện	iv
Lời cảm ơn	v
Đề cương	vii
Mục lục	viii
Danh sách hình	xi
Danh sách bảng	xii
Tóm tắt	xii
1 Giới thiệu	1
1.1 Bối cảnh và động lực nghiên cứu	1
1.2 Phát biểu bài toán	5
1.2.1 Đầu vào của hệ thống	5
1.2.2 Quá trình xử lý	6
1.2.3 Đầu ra của hệ thống	7
1.3 Các phương pháp tiếp cận phổ biến và hạn chế	8
1.3.1 Kiểm thử dựa trên kịch bản mã nguồn	8

1.3.2	Kiểm thử dựa trên ghi lại và phát lại thao tác . . .	10
1.4	Hướng tiếp cận đề xuất	11
1.4.1	Vì sao lựa chọn tác nhân kiểm thử dựa trên mô hình ngôn ngữ lớn	12
1.4.2	Hệ thống hoạt động như thế nào	13
1.5	Mục tiêu và phạm vi nghiên cứu	15
2	Cơ sở lý thuyết và các nghiên cứu liên quan	17
2.1	Tổng quan về kiểm thử phần mềm	18
2.1.1	Khái niệm	18
2.1.2	Các mức kiểm thử	18
2.1.3	Kiểm thử chức năng	18
2.1.4	Kiểm thử tự động	18
2.2	Các framework kiểm thử Web	18
2.2.1	Selenium	18
2.2.2	Playwright	18
2.2.3	So sánh các framework	18
2.3	Mô hình ngôn ngữ lớn (LLM)	18
2.3.1	Khái niệm	18
2.3.2	Prompt Engineering	18
2.3.3	Function Calling	18
2.3.4	Tool Calling	18
2.4	AI Agent	18
2.4.1	Khái niệm	18
2.4.2	Kiến trúc AI Agent	18
2.4.3	Quy trình hoạt động	18
2.5	Browser Automation	18
2.5.1	Browser-use	18
2.5.2	Playwright	18
2.5.3	Tương tác với DOM	18
2.5.4	Vision-based Interaction	18

2.6	Các công trình nghiên cứu liên quan	18
2.6.1	Trong nước	18
2.6.2	Quốc tế	18
2.6.3	So sánh và đánh giá	18
2.7	Kết chương	18
3	Hướng dẫn sử dụng template	19
3.1	Trích dẫn tài liệu	19
3.2	Chèn mã nguồn	19
3.3	Hình ảnh	20
3.4	Bảng biểu	21
3.5	Công thức	21
	Danh mục công trình của tác giả	23
	Tài liệu tham khảo	24
A	Ngữ pháp tiếng Việt	27
B	Ngữ pháp tiếng Nôm	28

Danh sách hình

3.1	Hình ví dụ 1	21
3.2	Hình ví dụ 2	21

Danh sách bảng

3.1	Bảng ví dụ 1	22
-----	------------------------	----

Tóm tắt

Giới hạn 200-300 từ, sử dụng văn phong học thuật, ngắn gọn, khách quan, viết liền mạch, không gạch đầu dòng.

Hướng dẫn trình bày tóm tắt:

1. Giới thiệu vấn đề/ngữ cảnh của vấn đề (khoảng 1 - 3 câu): Trình bày bối cảnh và vấn đề thực tế đang tồn tại/ khoảng trống nghiên cứu, tầm quan trọng của vấn đề nghiên cứu, lý do cần thực hiện đề tài.
2. Mục tiêu đề tài (1 câu): nêu cụ thể đề tài giải quyết/phát triển điều gì.
3. Phương pháp và giải pháp (1 - 3 câu): Trình bày phương pháp, công nghệ, công cụ, mô hình, giải pháp được đề xuất trong đề tài. Nêu những cải tiến nổi bật (nếu có). Nêu phương pháp thử nghiệm, đánh giá, bộ dữ liệu được sử dụng để đánh giá (nếu có)
4. Kết quả chính (1 - 2 câu): Trình bày ngắn gọn các kết quả nổi bật, có thể đưa số liệu cụ thể (độ chính xác, tốc độ xử lý, dung lượng dữ liệu, ...)
5. Kết luận (1 - 2 câu): Nêu ý nghĩa khoa học và thực tiễn (nếu có) của phương pháp/giải pháp hay kết quả của đề tài, khả năng áp dụng vào thực tế/đóng góp cho doanh nghiệp/người dùng/nghiên cứu sau này, ...

Chương 1

Giới thiệu

1.1 Bối cảnh và động lực nghiên cứu

Trong những năm gần đây, ứng dụng web đã trở thành một trong những nền tảng phần mềm phổ biến nhất để cung cấp dịch vụ số cho người dùng cá nhân, doanh nghiệp và tổ chức. Các hệ thống thương mại điện tử, ngân hàng số, giáo dục trực tuyến, quản lý nội bộ, chăm sóc khách hàng và nhiều loại phần mềm doanh nghiệp khác đều được triển khai thông qua trình duyệt web nhờ khả năng truy cập linh hoạt, không phụ thuộc nhiều vào thiết bị và dễ dàng cập nhật. Cùng với sự phát triển của kiến trúc ứng dụng hiện đại, đặc biệt là các ứng dụng một trang, giao diện web không còn là tập hợp các trang tĩnh đơn giản mà đã trở thành những hệ thống tương tác phức tạp, có nhiều trạng thái, dữ liệu bất đồng bộ, thành phần giao diện động và luồng nghiệp vụ thay đổi liên tục theo nhu cầu vận hành thực tế.

Tốc độ phát triển phần mềm hiện đại đặt ra yêu cầu ngày càng cao đối với hoạt động đảm bảo chất lượng. Trong mô hình phát triển truyền thống, phần mềm thường được kiểm thử sau khi hoàn thành một tập chức năng lớn. Cách làm này không còn phù hợp với môi trường phát triển linh hoạt, nơi các nhóm kỹ thuật thường xuyên cập nhật mã nguồn, tích hợp thay đổi và phát hành phiên bản mới trong chu kỳ ngắn. Mỗi thay đổi ở

mã nguồn, giao diện, cấu trúc dữ liệu hoặc quy trình nghiệp vụ đều có khả năng tạo ra lỗi hồi quy, nghĩa là một chức năng từng hoạt động đúng ở phiên bản trước có thể bị sai lệch ở phiên bản sau. Khi các lỗi này không được phát hiện kịp thời, chúng có thể ảnh hưởng trực tiếp đến trải nghiệm người dùng, uy tín sản phẩm và chi phí vận hành của tổ chức.

Kiểm thử phần mềm vì vậy không chỉ là hoạt động phát hiện lỗi trước khi bàn giao sản phẩm, mà còn là cơ chế kiểm soát rủi ro trong toàn bộ vòng đời phát triển. Nhiều nghiên cứu trong kỹ nghệ phần mềm đã chỉ ra rằng chi phí phát hiện và khắc phục lỗi tăng lên đáng kể khi lỗi được phát hiện muộn. Boehm và Basili cho rằng việc tìm và sửa lỗi sau khi phần mềm đã được bàn giao có thể tốn kém hơn nhiều lần so với việc phát hiện lỗi ở giai đoạn yêu cầu hoặc thiết kế [1]. Nhận định này cho thấy giá trị của việc phát hiện lỗi sớm, đặc biệt trong bối cảnh phần mềm web thường xuyên thay đổi và phục vụ số lượng lớn người dùng.

Trong thực tế phát triển phần mềm, kiểm thử thủ công vẫn đóng vai trò quan trọng vì con người có khả năng đánh giá ngữ cảnh, phát hiện bất thường về trải nghiệm và hiểu mục tiêu nghiệp vụ. Dù vậy, kiểm thử thủ công gặp nhiều hạn chế khi số lượng chức năng tăng lên và chu kỳ phát hành bị rút ngắn. Những ca kiểm thử hồi quy phải được thực hiện lặp lại sau mỗi lần cập nhật phần mềm, dẫn đến tiêu tốn nhiều thời gian, công sức và dễ phát sinh sai sót do yếu tố con người. Các luồng nghiệp vụ phổ biến như đăng nhập, đăng ký tài khoản, tìm kiếm sản phẩm, thêm vào giỏ hàng, thanh toán, cập nhật thông tin cá nhân hoặc kiểm tra quyền truy cập thường phải được kiểm tra nhiều lần ở các môi trường khác nhau. Khi quy mô hệ thống tăng lên, việc phụ thuộc hoàn toàn vào kiểm thử thủ công sẽ làm chậm tốc độ phát hành và làm giảm khả năng phát hiện lỗi kịp thời.

Kiểm thử tự động được xem là một hướng giải quyết quan trọng cho vấn đề này. Thay vì yêu cầu người kiểm thử thực hiện lặp lại từng thao tác trên giao diện, hệ thống kiểm thử tự động có thể mô phỏng hành vi người dùng, thực thi kịch bản kiểm thử, so sánh kết quả thực tế với kết

quả mong đợi và tạo báo cáo sau mỗi lần chạy. Các công cụ như Selenium WebDriver, Cypress và Playwright đã trở thành nền tảng quen thuộc trong kiểm thử ứng dụng web. Selenium cung cấp khả năng tự động hóa trình duyệt phục vụ kiểm thử ứng dụng web [2]. Playwright hỗ trợ điều khiển nhiều trình duyệt như Chromium, Firefox và WebKit thông qua một API thống nhất [3]. Những công cụ này giúp kiểm thử tự động có thể tích hợp vào quy trình tích hợp liên tục và triển khai liên tục, từ đó hỗ trợ nhóm phát triển phát hiện lỗi sớm hơn trong quá trình thay đổi phần mềm.

Sự phát triển của kiểm thử tự động cũng được phản ánh qua quy mô thị trường. Theo Research Nester, thị trường automation testing toàn cầu được ước tính đạt 42,32 tỷ USD trong năm 2026 và có thể tiếp tục tăng trưởng trong giai đoạn sau đó [4]. Con số này cho thấy kiểm thử tự động không còn là một lựa chọn phụ trợ mà đã trở thành một thành phần quan trọng trong chiến lược đảm bảo chất lượng phần mềm. Song song với đó, World Quality Report 2025 ghi nhận sự gia tăng mạnh của các quy trình quality engineering có sử dụng trí tuệ nhân tạo sinh, khi phần lớn tổ chức được khảo sát đã bắt đầu thí điểm hoặc triển khai các workflow có tích hợp GenAI [5]. Xu hướng này cho thấy ngành kiểm thử phần mềm đang dịch chuyển từ tự động hóa dựa trên kịch bản cố định sang các phương pháp có khả năng hỗ trợ suy luận, phân tích và thích nghi tốt hơn.

Dù kiểm thử tự động đã mang lại nhiều lợi ích, việc xây dựng và duy trì kịch bản kiểm thử cho ứng dụng web vẫn còn nhiều khó khăn. Phần lớn công cụ kiểm thử hiện tại yêu cầu kỹ sư kiểm thử hoặc lập trình viên mô tả thao tác thông qua mã nguồn, bộ chọn phần tử, đường dẫn XPath, CSS selector hoặc các thuộc tính cụ thể trong cây DOM. Cách tiếp cận này có độ chính xác cao khi giao diện ổn định, nhưng dễ bị ảnh hưởng khi cấu trúc HTML, tên lớp CSS, bố cục giao diện hoặc thư viện thành phần thay đổi. Trong nhiều trường hợp, kịch bản kiểm thử thất bại không phải vì ứng dụng có lỗi nghiệp vụ, mà vì bộ chọn phần tử không còn phù hợp với phiên bản giao diện mới. Đây là vấn đề thường được gọi là kịch bản kiểm thử giòn.

Vấn đề càng trở nên rõ ràng hơn đối với các ứng dụng web hiện đại. Giao diện có thể thay đổi theo trạng thái đăng nhập, quyền người dùng, dữ liệu trả về từ máy chủ, ngôn ngữ hiển thị, kích thước màn hình hoặc lịch sử tương tác. Một thành phần có thể được tải sau khi gọi API, một nút bấm có thể xuất hiện sau khi người dùng hoàn thành điều kiện nhất định, hoặc cùng một chức năng có thể được biểu diễn bằng bố cục khác nhau ở các phiên bản giao diện khác nhau. Các nghiên cứu về flaky UI tests cũng cho thấy kiểm thử giao diện có không gian trạng thái và điều kiện chạy phức tạp hơn so với kiểm thử đơn vị, khiến việc phát hiện, tái hiện và sửa lỗi kiểm thử trở nên khó khăn hơn [6].

Từ thực tế trên, có thể thấy khoảng cách giữa mục tiêu nghiệp vụ của ca kiểm thử và cách biểu diễn kỹ thuật của kịch bản kiểm thử là một nguyên nhân quan trọng tạo ra chi phí bảo trì. Người kiểm thử thường muốn mô tả mục tiêu ở mức ngữ nghĩa, chẳng hạn như đăng nhập bằng tài khoản hợp lệ rồi kiểm tra trang chủ hiển thị đúng, trong khi công cụ kiểm thử truyền thống lại yêu cầu mô tả ở mức kỹ thuật, chẳng hạn như tìm ô nhập liệu theo selector, nhấn nút theo XPath và kiểm tra URL cụ thể. Khi giao diện thay đổi nhưng mục tiêu nghiệp vụ vẫn giữ nguyên, kịch bản kiểm thử đáng lẽ vẫn có thể tiếp tục được sử dụng. Thực tế, kịch bản thường bị hỏng vì bị neo cứng vào biểu diễn kỹ thuật của giao diện tại một thời điểm nhất định.

Sự xuất hiện của các mô hình ngôn ngữ lớn và các mô hình đa phương thức mở ra một hướng tiếp cận mới cho bài toán kiểm thử tự động ứng dụng web. Gemini API của Google hỗ trợ xử lý ngôn ngữ tự nhiên và hiểu nội dung từ các dữ liệu phi cấu trúc như hình ảnh, tài liệu và video [7]. Khi kết hợp khả năng hiểu ngôn ngữ, hiểu ngữ cảnh giao diện và điều khiển trình duyệt, một tác nhân AI có thể tiếp nhận ca kiểm thử được mô tả bằng ngôn ngữ tự nhiên, quan sát trạng thái hiện tại của ứng dụng, suy luận hành động tiếp theo và thực thi thao tác trên trình duyệt. Hướng tiếp cận này giúp chuyển tri thức kiểm thử từ tầng kỹ thuật sang tầng ngữ nghĩa, nơi ca kiểm thử được mô tả gần hơn với cách con người hiểu yêu

cầu nghiệp vụ.

Chính vì vậy, tôi quyết định chọn đề tài “Xây dựng hệ thống kiểm thử tự động ứng dụng web” làm chủ đề cho khóa luận này.

1.2 Phát biểu bài toán

Bài toán đặt ra trong khóa luận là xây dựng một hệ thống kiểm thử tự động cho ứng dụng web có khả năng tiếp nhận kịch bản kiểm thử bằng ngôn ngữ tự nhiên, quan sát trạng thái giao diện đang chạy trên trình duyệt, tự động suy luận hành động cần thực hiện và tạo báo cáo kết quả sau khi hoàn thành. Hệ thống hướng đến việc giảm sự phụ thuộc vào kịch bản kiểm thử được viết cứng bằng mã nguồn hoặc selector, đồng thời hỗ trợ người dùng không chuyên sâu về lập trình có thể mô tả ca kiểm thử ở mức nghiệp vụ.

1.2.1 Đầu vào của hệ thống

Đầu vào của hệ thống bao gồm kịch bản kiểm thử, thông tin ứng dụng cần kiểm thử và trạng thái quan sát được tại thời điểm thực thi. Kịch bản kiểm thử là thành phần quan trọng nhất vì nó thể hiện mục tiêu nghiệp vụ mà người dùng mong muốn xác minh. Kịch bản này được mô tả bằng ngôn ngữ tự nhiên, không yêu cầu người dùng viết mã nguồn, khai báo selector hoặc hiểu cấu trúc DOM của ứng dụng. Mỗi kịch bản có thể bao gồm một chuỗi hành động, một hoặc nhiều điều kiện kiểm tra và tiêu chí để xác định ca kiểm thử đạt hay không đạt.

Thông tin ứng dụng cần kiểm thử bao gồm địa chỉ URL điểm vào, cấu hình môi trường, tài khoản kiểm thử nếu cần và các tham số liên quan đến phiên chạy. Đối với những luồng nghiệp vụ yêu cầu xác thực, hệ thống có thể cần thông tin tài khoản hoặc trạng thái đăng nhập đã được chuẩn bị trước. Đối với những luồng kiểm thử khác, hệ thống chỉ cần URL ban đầu và mô tả mục tiêu để bắt đầu thực thi.

Trạng thái quan sát của ứng dụng bao gồm cây DOM hiện tại, ảnh chụp màn hình, thông tin phần tử có thể tương tác và các dữ liệu hỗ trợ khác. DOM cung cấp cấu trúc kỹ thuật của trang, cho biết các phần tử HTML, thuộc tính, quan hệ cha con và trạng thái hiển thị. Ảnh chụp màn hình cung cấp ngữ cảnh thị giác, giúp tác nhân nhận biết bố cục, văn bản hiển thị, vị trí tương đối của các thành phần và các dấu hiệu trực quan mà DOM có thể không biểu diễn đầy đủ. Việc kết hợp hai nguồn thông tin này giúp hệ thống có cái nhìn đầy đủ hơn về giao diện đang được kiểm thử.

1.2.2 Quá trình xử lý

Quá trình xử lý của hệ thống được tổ chức theo vòng lặp quan sát, suy luận và hành động. Tại mỗi bước, hệ thống thu thập trạng thái hiện tại của trình duyệt, bao gồm DOM đã được chú thích và ảnh chụp màn hình của trang. Trạng thái này được kết hợp với mô tả kịch bản kiểm thử và lịch sử các bước đã thực hiện để tạo thành ngữ cảnh đầu vào cho mô hình ngôn ngữ lớn.

Sau khi nhận ngữ cảnh, mô hình phân tích mục tiêu kiểm thử, xác định trạng thái hiện tại đang ở đâu trong luồng nghiệp vụ và suy luận hành động tiếp theo cần thực hiện. Hành động có thể là nhập dữ liệu vào ô văn bản, nhấn nút, chọn giá trị trong danh sách, cuộn trang, điều hướng đến một khu vực cụ thể hoặc kiểm tra một điều kiện hiển thị. Kết quả suy luận cần được biểu diễn dưới dạng có cấu trúc để thành phần thực thi có thể chuyển thành thao tác trên trình duyệt.

Thành phần thực thi nhận quyết định từ mô hình và thao tác trên trình duyệt thông qua công cụ tự động hóa. Trong khóa luận này, Playwright được sử dụng như thành phần thực thi vì công cụ này hỗ trợ điều khiển nhiều trình duyệt và phù hợp với kiểm thử ứng dụng web hiện đại [3]. Sau mỗi hành động, hệ thống chụp lại trạng thái mới của giao diện, ghi nhận bằng chứng kiểm thử và tiếp tục vòng lặp. Quá trình này kết thúc khi kịch

bản hoàn thành, điều kiện kiểm tra được xác nhận, hoặc hệ thống phát hiện lỗi khiến ca kiểm thử không thể tiếp tục.

1.2.3 Đầu ra của hệ thống

Đầu ra của hệ thống bao gồm trạng thái kiểm thử, báo cáo thực thi và thông tin phân tích lỗi. Trạng thái kiểm thử cho biết kết quả tổng quát của ca kiểm thử sau khi chạy. Một ca kiểm thử có thể được đánh dấu là đạt khi toàn bộ hành động và điều kiện kiểm tra được hoàn thành đúng như mô tả. Ca kiểm thử được đánh dấu là thất bại khi hệ thống thực thi đúng kịch bản nhưng kết quả của ứng dụng không thỏa mãn tiêu chí mong đợi. Trường hợp lỗi thực thi được sử dụng khi tác nhân không thể hoàn thành ca kiểm thử do vấn đề kỹ thuật, chẳng hạn như trang không tải được, trình duyệt gặp lỗi, mô hình không đưa ra hành động hợp lệ hoặc môi trường kiểm thử không ổn định.

Báo cáo thực thi là thành phần quan trọng giúp người dùng kiểm chứng kết quả. Báo cáo cần ghi lại chuỗi hành động mà tác nhân đã thực hiện, thời điểm thực hiện, ảnh chụp màn hình sau mỗi bước, nhật ký thao tác và kết quả đánh giá tại các điểm xác nhận. Các bằng chứng này giúp người kiểm thử hoặc lập trình viên có thể xem lại quá trình chạy, xác định bước xảy ra lỗi và so sánh kết quả giữa các lần thực thi khác nhau.

Thông tin phân tích lỗi hỗ trợ người dùng hiểu nguyên nhân thất bại của ca kiểm thử. Khi kết quả không đạt, hệ thống có thể sử dụng mô hình ngôn ngữ lớn để phân tích sự khác biệt giữa mục tiêu kiểm thử, hành động đã thực hiện và trạng thái thực tế của giao diện. Kết quả phân tích có thể gợi ý rằng ứng dụng có lỗi nghiệp vụ, giao diện thay đổi khiến ca kiểm thử cần cập nhật, dữ liệu đầu vào không còn hợp lệ hoặc tác nhân đã chọn sai phần tử tương tác. Đây là cơ sở để phát triển các cơ chế tự sửa hoặc đề xuất cập nhật kịch bản trong các phiên bản mở rộng của hệ thống.

1.3 Các phương pháp tiếp cận phổ biến và hạn chế

Để làm rõ đóng góp của hướng tiếp cận được đề xuất, cần xem xét các phương pháp kiểm thử tự động ứng dụng web đang được sử dụng phổ biến. Hai nhóm tiếp cận tiêu biểu là kiểm thử dựa trên kịch bản mã nguồn và kiểm thử dựa trên ghi lại thao tác. Mỗi phương pháp đều có ưu điểm riêng và đã được áp dụng rộng rãi trong thực tế. Dù vậy, cả hai vẫn gặp khó khăn khi ứng dụng web thay đổi thường xuyên, giao diện có tính động cao và người dùng muốn mô tả kiểm thử ở mức nghiệp vụ thay vì mức kỹ thuật.

1.3.1 Kiểm thử dựa trên kịch bản mã nguồn

Kiểm thử dựa trên kịch bản mã nguồn là phương pháp phổ biến và trưởng thành nhất trong kiểm thử tự động ứng dụng web. Với phương pháp này, kỹ sư kiểm thử hoặc lập trình viên viết mã để mô tả từng bước tương tác với trình duyệt. Công cụ kiểm thử sẽ mở trang web, tìm phần tử giao diện thông qua selector hoặc cơ chế định vị, thực hiện thao tác người dùng và kiểm tra kết quả mong đợi. Selenium WebDriver là một đại diện tiêu biểu của hướng tiếp cận này, với khả năng tự động hóa trình duyệt phục vụ kiểm thử ứng dụng web [2]. Cypress và Playwright cũng được sử dụng rộng rãi cho kiểm thử giao diện và kiểm thử đầu cuối của ứng dụng web hiện đại.

Ưu điểm lớn nhất của phương pháp này là tính rõ ràng và khả năng kiểm soát cao. Mỗi bước kiểm thử được mô tả cụ thể bằng mã nguồn nên người viết có thể điều chỉnh logic chờ đợi, dữ liệu đầu vào, điều kiện kiểm tra và cách xử lý ngoại lệ. Kịch bản kiểm thử có thể được đưa vào hệ thống quản lý mã nguồn, chạy tự động trong quy trình CI/CD và tái sử dụng nhiều lần. Đối với các hệ thống có giao diện ổn định và đội ngũ kỹ thuật đủ năng lực, kiểm thử dựa trên mã nguồn mang lại hiệu quả cao

trong việc phát hiện lỗi hồi quy.

Hạn chế cốt lõi của phương pháp này nằm ở sự phụ thuộc vào biểu diễn kỹ thuật của giao diện. Để thao tác với một phần tử, kịch bản thường phải xác định phần tử đó thông qua CSS selector, XPath, thuộc tính HTML hoặc một cơ chế định vị tương tự. Khi nhóm phát triển thay đổi cấu trúc DOM, đổi tên lớp CSS, thay đổi thư viện giao diện hoặc điều chỉnh bố cục, kịch bản kiểm thử có thể thất bại dù chức năng nghiệp vụ vẫn hoạt động đúng. Lỗi lúc này không phản ánh chất lượng của ứng dụng mà phản ánh sự không tương thích giữa kịch bản kiểm thử và phiên bản giao diện mới.

Một hạn chế khác là chi phí xây dựng và bảo trì. Việc viết kịch bản kiểm thử đòi hỏi người thực hiện có hiểu biết về lập trình, cấu trúc web, công cụ kiểm thử và quy trình phát triển phần mềm. Điều này tạo ra rào cản đối với các vai trò hiểu nghiệp vụ nhưng không chuyên về kỹ thuật, chẳng hạn như kiểm thử viên thủ công, chuyên viên phân tích nghiệp vụ hoặc chủ sở hữu sản phẩm. Trong nhiều dự án, những người hiểu rõ nhất về kỳ vọng nghiệp vụ lại không trực tiếp viết được ca kiểm thử tự động, còn những người viết được mã kiểm thử lại phải diễn giải yêu cầu nghiệp vụ thông qua tài liệu trung gian. Khoảng cách này làm tăng chi phí giao tiếp và có thể làm giảm độ chính xác của kịch bản kiểm thử.

Kiểm thử giao diện cũng dễ gặp hiện tượng không ổn định. Romano và cộng sự chỉ ra rằng UI tests có không gian đầu vào rộng hơn, điều kiện chạy đa dạng hơn và chi phí thực thi cao hơn so với kiểm thử đơn vị, khiến việc phát hiện và xử lý flaky tests trở nên phức tạp hơn [6]. Trong môi trường thực tế, một ca kiểm thử có thể thất bại do thời gian tải chậm, dữ liệu bất đồng bộ chưa sẵn sàng, trạng thái trình duyệt khác nhau hoặc thành phần giao diện xuất hiện muộn. Những yếu tố này làm cho việc duy trì bộ kiểm thử tự động trở thành một công việc liên tục, đặc biệt khi sản phẩm phát triển nhanh.

1.3.2 Kiểm thử dựa trên ghi lại và phát lại thao tác

Kiểm thử dựa trên ghi lại và phát lại thao tác được phát triển nhằm giảm rào cản kỹ thuật của phương pháp viết mã kiểm thử thủ công. Với cách tiếp cận này, người dùng thực hiện một luồng thao tác trên trình duyệt, công cụ ghi lại các hành động đó và sinh ra kịch bản kiểm thử tương ứng. Khi cần kiểm thử lại, công cụ sẽ phát lại chuỗi thao tác đã ghi và so sánh kết quả với trạng thái mong đợi. Cách làm này phù hợp với người dùng không có nhiều kinh nghiệm lập trình vì họ có thể tạo kịch bản kiểm thử bằng chính thao tác sử dụng ứng dụng.

Ưu điểm của phương pháp này là tốc độ tạo kịch bản ban đầu nhanh. Người kiểm thử có thể ghi lại một luồng đăng nhập, tìm kiếm hoặc đặt hàng mà không cần viết từng dòng mã. Một số công cụ low code còn cung cấp giao diện trực quan để chỉnh sửa bước kiểm thử, thêm điều kiện kiểm tra và tổ chức bộ test case. Cách tiếp cận này phù hợp cho kiểm thử khối, kiểm thử nhanh các luồng ổn định hoặc tạo bản nháp ban đầu cho kịch bản kiểm thử tự động.

Hạn chế của phương pháp ghi lại và phát lại nằm ở chất lượng kịch bản được sinh ra. Do công cụ ghi lại thao tác ở mức giao diện, kịch bản thường phụ thuộc mạnh vào vị trí phần tử, XPath, selector hoặc trạng thái cụ thể tại thời điểm ghi. Khi giao diện thay đổi, dữ liệu hiển thị khác đi hoặc luồng tương tác có thêm điều kiện mới, kịch bản có thể không còn phù hợp. Trong nhiều trường hợp, kịch bản được sinh tự động còn khó đọc và khó bảo trì hơn kịch bản do con người viết vì nó chứa nhiều chi tiết kỹ thuật không phản ánh rõ mục tiêu nghiệp vụ.

Phương pháp này cũng khó xử lý các tình huống động trong ứng dụng web hiện đại. Một luồng thao tác được ghi lại có thể hoạt động đúng với một bộ dữ liệu cụ thể nhưng thất bại khi dữ liệu thay đổi. Ví dụ, vị trí của một sản phẩm trong danh sách có thể khác giữa các lần chạy, thông báo xác nhận có thể hiển thị chậm hơn, hoặc nút thao tác chỉ xuất hiện sau khi người dùng có quyền phù hợp. Nếu công cụ chỉ phát lại cứng nhắc

những thao tác đã ghi, nó khó thích nghi với biến thể giao diện và trạng thái ứng dụng.

Từ hai nhóm phương pháp trên, có thể rút ra một giới hạn chung. Phần lớn công cụ kiểm thử tự động hiện nay biểu diễn tri thức kiểm thử ở tầng kỹ thuật, thông qua mã nguồn, selector, DOM hoặc chuỗi thao tác cố định. Trong khi đó, mục tiêu thật sự của kiểm thử thường nằm ở tầng nghiệp vụ. Người kiểm thử muốn xác minh rằng người dùng có thể đăng nhập, thêm sản phẩm vào giỏ hàng hoặc hoàn tất thanh toán, chứ không chỉ muốn nhấn vào một phần tử có XPath cụ thể. Khi mục tiêu nghiệp vụ ổn định nhưng giao diện thay đổi, một hệ thống kiểm thử lý tưởng cần có khả năng nhận ra ý nghĩa của hành động và thích nghi với biến thể giao diện. Đây chính là khoảng trống mà hướng tiếp cận dựa trên tác nhân AI và mô hình ngôn ngữ lớn hướng đến giải quyết.

1.4 Hướng tiếp cận đề xuất

Khóa luận đề xuất xây dựng hệ thống kiểm thử tự động ứng dụng web dựa trên tác nhân AI có khả năng tiếp nhận kịch bản kiểm thử bằng ngôn ngữ tự nhiên, quan sát giao diện thông qua DOM và ảnh chụp màn hình, sử dụng mô hình ngôn ngữ lớn để suy luận hành động và thực thi thao tác trên trình duyệt bằng công cụ tự động hóa. Thay vì yêu cầu người dùng mô tả từng bước bằng mã nguồn, hệ thống cho phép mô tả mục tiêu kiểm thử ở mức gần với ngôn ngữ nghiệp vụ. Tác nhân kiểm thử sẽ đảm nhận việc chuyển đổi mục tiêu đó thành các hành động cụ thể trên giao diện.

Hướng tiếp cận này dựa trên nhận định rằng tri thức kiểm thử nên được biểu diễn ở tầng ngữ nghĩa thay vì tầng kỹ thuật. Khi một kịch bản được viết dưới dạng “đăng nhập bằng tài khoản hợp lệ và kiểm tra trang chính hiển thị”, nội dung quan trọng là mục tiêu đăng nhập và điều kiện xác nhận sau đăng nhập. Selector của ô email, XPath của nút đăng nhập hoặc cấu trúc HTML chỉ là biểu diễn kỹ thuật tại một thời điểm. Nếu biểu diễn kỹ thuật thay đổi nhưng mục tiêu nghiệp vụ giữ nguyên, hệ thống

nên có khả năng tiếp tục thực hiện kiểm thử bằng cách quan sát giao diện mới và suy luận phần tử tương ứng.

1.4.1 Vì sao lựa chọn tác nhân kiểm thử dựa trên mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn phù hợp với bài toán này vì có khả năng xử lý mô tả tự nhiên và suy luận theo ngữ cảnh. Trong kiểm thử phần mềm, kịch bản thường được mô tả bằng ngôn ngữ gần với nghiệp vụ, chẳng hạn như kiểm tra người dùng có thể đăng nhập, xác nhận sản phẩm được thêm vào giỏ hàng hoặc đảm bảo thông báo lỗi xuất hiện khi nhập dữ liệu không hợp lệ. Những mô tả này không nhất thiết phải ánh xạ trực tiếp đến một selector duy nhất, mà cần được hiểu trong bối cảnh giao diện hiện tại. Mô hình ngôn ngữ lớn có thể hỗ trợ phân tích mục tiêu, xác định bước tiếp theo và đưa ra hành động phù hợp dựa trên trạng thái quan sát được.

Khả năng đa phương thức làm cho hướng tiếp cận này trở nên phù hợp hơn với kiểm thử giao diện. Gemini API hỗ trợ hiểu dữ liệu hình ảnh và văn bản, cho phép mô hình tiếp nhận ảnh chụp màn hình cùng với mô tả nhiệm vụ [7]. Trong kiểm thử ứng dụng web, nhiều thông tin quan trọng nằm ở bố cục hiển thị, nhãn nút, thông báo lỗi, biểu tượng, vị trí tương đối và trạng thái trực quan của thành phần giao diện. Khi kết hợp ảnh chụp màn hình với DOM, tác nhân có thể có cả góc nhìn thị giác và góc nhìn cấu trúc, từ đó tăng khả năng đưa ra hành động đúng.

Hướng tiếp cận dựa trên tác nhân AI cũng có tiềm năng giảm rào cản tham gia kiểm thử tự động. Người kiểm thử thủ công, chuyên viên phân tích nghiệp vụ hoặc chủ sở hữu sản phẩm có thể mô tả ca kiểm thử bằng ngôn ngữ tự nhiên thay vì viết mã. Điều này không loại bỏ vai trò của kỹ sư kiểm thử tự động, mà giúp phân tách rõ hơn giữa tri thức nghiệp vụ và cơ chế thực thi kỹ thuật. Kỹ sư kiểm thử có thể tập trung vào thiết kế hệ thống, cấu hình môi trường, đánh giá độ tin cậy và tối ưu quy trình, trong khi người hiểu nghiệp vụ có thể trực tiếp đóng góp vào nội dung kịch bản

kiểm thử.

Báo cáo World Quality Report 2025 cho thấy GenAI đang được quan tâm mạnh trong lĩnh vực quality engineering, nhưng việc triển khai ở quy mô doanh nghiệp vẫn còn nhiều thách thức [5]. Điều này cho thấy hướng nghiên cứu tác nhân AI cho kiểm thử tự động vừa có tính thực tiễn, vừa cần được đánh giá cẩn thận. Hệ thống không nên được nhìn nhận như một công cụ thay thế hoàn toàn các phương pháp kiểm thử hiện tại, mà như một hướng bổ sung nhằm giải quyết những điểm yếu của kiểm thử dựa trên selector và kịch bản cứng.

1.4.2 Hệ thống hoạt động như thế nào

Về mặt tổng thể, hệ thống được xây dựng dưới dạng nền tảng web đa tầng, bao gồm giao diện người dùng xây dựng trên React, máy chủ ứng dụng sử dụng Node.js và Express để quản lý ca kiểm thử và lưu trữ kết quả qua PostgreSQL, và một dịch vụ tác nhân độc lập viết bằng Python với FastAPI phụ trách toàn bộ quá trình thực thi trên trình duyệt. Kiến trúc tách biệt giữa máy chủ ứng dụng và dịch vụ tác nhân cho phép hai thành phần triển khai độc lập, giao tiếp qua cơ chế callback không đồng bộ sau mỗi bước thực thi.

Trong dịch vụ tác nhân, cơ chế thực thi được tổ chức theo ba lớp: lớp điều phối, lớp suy luận và lớp thực thi trình duyệt. Lớp điều phối sử dụng thư viện browser-use để quản lý vòng lặp kiểm thử, trích xuất trạng thái trình duyệt, chuẩn hóa dữ liệu đầu vào cho mô hình và ghi lại bằng chứng sau mỗi bước [8]. Lớp suy luận sử dụng mô hình ngôn ngữ lớn để phân tích mục tiêu kiểm thử và xác định hành động tiếp theo dựa trên DOM đã xử lý, ảnh chụp màn hình và lịch sử thao tác. Về mặt thiết kế, thành phần suy luận được xây dựng theo hướng có thể thay thế mô hình, trong đó Gemini được sử dụng làm mô hình chính trong phạm vi đánh giá của khóa luận. Lớp thực thi sử dụng Playwright để điều khiển trình duyệt trực tiếp, hỗ trợ Chromium, Firefox và WebKit [3].

Hệ thống cung cấp hai chế độ thực thi: chế độ tác nhân AI và chế độ phát lại xác định. Trong chế độ tác nhân AI, browser-use Agent tiếp nhận mục tiêu kiểm thử bằng ngôn ngữ tự nhiên và tự động điều hướng trình duyệt qua vòng lặp quan sát – suy luận – hành động. Ở bước quan sát, hệ thống thu thập trạng thái hiện tại của trang; ở bước suy luận, mô hình xác định hành động phù hợp với mục tiêu kiểm thử; ở bước hành động, Playwright thực thi thao tác và ghi nhận kết quả. Sau khi chạy thành công, hệ thống tự động chuyển đổi lịch sử thao tác của tác nhân thành kịch bản JSON có cấu trúc để sử dụng cho các lần kiểm thử tiếp theo. Trong chế độ phát lại xác định, Playwright trực tiếp thực thi kịch bản JSON đã ghi mà không cần gọi mô hình ngôn ngữ, giúp kiểm thử hồi quy nhanh hơn và ổn định hơn. Khi một locator không tìm thấy phần tử, hệ thống áp dụng cơ chế thử lại với các locator thay thế, nhằm tăng khả năng phục hồi của kịch bản trước một số thay đổi nhỏ của giao diện.

Sau mỗi lần chạy ở chế độ tác nhân AI, hệ thống thực hiện thêm một lần gọi mô hình để tổng hợp các điều kiện xác nhận trạng thái giao diện cho từng bước, gọi là state anchor. Mỗi anchor mô tả một điều kiện có thể kiểm chứng tự động trong lần phát lại tiếp theo, chẳng hạn như URL phải chứa một chuỗi cụ thể, một văn bản nhất định phải xuất hiện trên trang, hoặc không có thông báo lỗi nào hiển thị. Cơ chế này giúp kịch bản phát lại không chỉ thực thi đúng thao tác mà còn xác nhận được trạng thái ứng dụng tại từng điểm quan trọng, mà không đòi hỏi người dùng viết điều kiện kiểm tra thủ công.

Hệ thống ghi lại bằng chứng kiểm thử sau mỗi bước thực thi, bao gồm ảnh chụp màn hình, mô tả hành động, suy luận của mô hình và nhật ký liên quan. Khi ca kiểm thử thất bại, mô hình ngôn ngữ lớn được gọi thêm một lần để phân tích nguyên nhân dựa trên lịch sử thực thi, giúp người dùng phân biệt lỗi do ứng dụng, lỗi do dữ liệu, lỗi do môi trường hay lỗi do tác nhân suy luận sai.

1.5 Mục tiêu và phạm vi nghiên cứu

Mục tiêu của khóa luận là xây dựng một hệ thống kiểm thử tự động ứng dụng web có khả năng thực thi ca kiểm thử dựa trên mô tả ngôn ngữ tự nhiên. Hệ thống cần cung cấp giao diện quản lý kịch bản kiểm thử, cơ chế lưu trữ kết quả, thành phần tác nhân thực thi và báo cáo bằng chứng sau mỗi lần chạy. Trọng tâm của khóa luận không chỉ nằm ở việc chạy được một số thao tác tự động trên trình duyệt, mà còn nằm ở việc thiết kế một quy trình kiểm thử trong đó mô hình ngôn ngữ lớn có thể tham gia vào quá trình suy luận hành động và phân tích kết quả.

Mục tiêu tiếp theo là đánh giá khả năng áp dụng của tác nhân AI trên các luồng nghiệp vụ phổ biến của ứng dụng web. Các luồng này có thể bao gồm đăng nhập, đăng ký, tìm kiếm, quản lý giỏ hàng, kiểm tra thông tin hiển thị và thực hiện các thao tác cơ bản trong quy trình mua hàng. Những luồng này được lựa chọn vì chúng đại diện cho các tương tác thường gặp trong ứng dụng web và có sự kết hợp giữa nhập liệu, nhấn nút, điều hướng, quan sát kết quả và xác nhận điều kiện.

Khóa luận cũng hướng đến việc giảm chi phí bảo trì kịch bản kiểm thử thông qua cách biểu diễn ca kiểm thử bằng ngôn ngữ tự nhiên. Khi kịch bản không bị ràng buộc chặt vào selector hoặc cấu trúc DOM cụ thể, hệ thống có thể thích nghi tốt hơn trước các thay đổi nhỏ của giao diện. Mặc dù không thể đảm bảo tác nhân luôn xử lý đúng mọi biến thể phức tạp, hướng tiếp cận này tạo cơ sở cho các cơ chế tự sửa và gợi ý cập nhật kịch bản trong tương lai.

Về phạm vi nghiên cứu, khóa luận tập trung vào kiểm thử giao diện người dùng của ứng dụng web chạy trên trình duyệt máy tính để bàn. Hệ thống sử dụng mô hình Gemini làm thành phần suy luận chính và Playwright làm thành phần thực thi trình duyệt. Các thử nghiệm chính được thực hiện trên trình duyệt Chromium, trong khi khả năng mở rộng sang Firefox và WebKit được xem xét thông qua hỗ trợ của Playwright. Khóa luận không đi sâu vào kiểm thử ứng dụng di động, kiểm thử hiệu

năng, kiểm thử bảo mật hoặc kiểm thử API độc lập. Những nội dung này có thể được xem là hướng mở rộng trong các nghiên cứu tiếp theo.

Tóm lại, chương này đã trình bày bối cảnh hình thành đề tài, phát biểu bài toán, đầu vào và đầu ra của hệ thống, các phương pháp tiếp cận phổ biến cùng hạn chế, cũng như lý do lựa chọn hướng tiếp cận dựa trên tác nhân AI và mô hình ngôn ngữ lớn. Từ các phân tích trên, đề tài hướng đến việc xây dựng một hệ thống kiểm thử tự động có khả năng hiểu yêu cầu ở mức ngôn ngữ tự nhiên, quan sát giao diện web, suy luận hành động và cung cấp báo cáo kiểm thử có thể truy vết. Đây là nền tảng để các chương tiếp theo trình bày cơ sở lý thuyết, kiến trúc hệ thống, quá trình hiện thực và kết quả đánh giá.

Chương 2

Cơ sở lý thuyết và các nghiên cứu liên quan

2.1 Tổng quan về kiểm thử phần mềm

2.1.1 Khái niệm

2.1.2 Các mức kiểm thử

Unit Test

Integration Test

System Test

Acceptance Test

2.1.3 Kiểm thử chức năng

2.1.4 Kiểm thử tự động

2.2 Các framework kiểm thử Web

2.2.1 Selenium

2.2.2 Playwright

18

2.2.3 So sánh các framework

2.3 Mô hình ngôn ngữ lớn (LLM)

Chương 3

Hướng dẫn sử dụng template

3.1 Trích dẫn tài liệu

Dùng lệnh `\cite` để trích dẫn một hoặc nhiều tài liệu tham khảo. Tài liệu tham khảo có thể là trang web [9], [10], bài báo khoa học [11], sách [12], [13], [14], bài tạp chí [15] hoặc các nguồn tham khảo khác. Lưu ý khi trích dẫn tài liệu tham khảo, cần viết câu sao cho bỏ phần trong cặp ngoặc vuông đi thì câu vẫn đầy đủ ý nghĩa. Ví dụ, thay vì viết “Nghiên cứu [15] chỉ ra rằng ... ” thì nên viết “Nghiên cứu của Zhang [15] chỉ ra rằng ...”. Một ví dụ khác, thay vì viết “... như trong công trình nghiên cứu [11].” thì nên viết “... như trong công trình nghiên cứu của Cavnar và Trenkle [11].”

3.2 Chèn mã nguồn

Để chèn mã nguồn, cần dùng package listings [9]:

```
1 \usepackage{listings}
```

Mã nguồn có thể được chèn trực tiếp như sau:

```
1 print "Hello , World!"
```

hoặc chèn thông qua tập tin chứa mã nguồn trong thư mục *SourceCode* như sau:

```
1 #include <iostream.h>
```

```

2
3 main()
4 {
5     for (;;)
6     {
7         cout << "Hello World! ";
8     }
9 }

```

Để chèn mã giả, cần dùng package algorithm **Algorithm**:

```

1 \usepackage{algorithm}

```

Có thể chèn mã giả vào như sau:

Algorithm 1 My algorithm

```

1: procedure MYPROCEDURE
2:    $stringlen \leftarrow \text{length of } string$ 
3:    $i \leftarrow patlen$ 
4: top:
5:   if  $i > stringlen$  then return false
6:    $j \leftarrow patlen$ 
7: loop:
8:   if  $string(i) = path(j)$  then
9:      $j \leftarrow j - 1.$ 
10:     $i \leftarrow i - 1.$ 
11:    goto loop.
12:   close;
13:    $i \leftarrow i + \max(delta_1(string(i)), delta_2(j)).$ 
14:   goto top.

```

3.3 Hình ảnh

Để chèn hình ảnh, cần dùng package graphicx [16]:

```

1 \usepackage{graphicx}

```

Hình 3.1, hình 3.2 là một số ví dụ về chèn hình ảnh.



Hình 3.1: Hình ví dụ 1



Hình 3.2: Hình ví dụ 2

3.4 Bảng biểu

Để tạo bảng biểu, tham khảo thêm tại sharelatex.com [17]. Bảng 3.1 là một ví dụ về bảng. Ngoài ra, có một số tool online ¹ có thể được dùng để tạo bảng biểu một cách trực quan.

3.5 Công thức

Công thức có thể chèn vào trong cùng một dòng như $\sqrt{a^2 + b^2}$ hoặc nằm trên dòng riêng như công thức 3.1.

$$x = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{a_3 + a_4}}} \quad (3.1)$$

¹Như trang <http://www.tablesgenerator.com/>

Bảng 3.1: Bảng ví dụ 1

Country List			
Country Name or Area Name	ISO ALPHA 2 Code	ISO ALPHA 3 Code	ISO numeric Code
Afghanistan	AF	AFG	004
Aland Islands	AX	ALA	248
Albania	AL	ALB	008
Algeria	DZ	DZA	012
American Samoa	AS	ASM	016
Andorra	AD	AND	020
Angola	AO	AGO	024

Danh mục công trình của tác giả

1. Tạp chí ABC
2. Tạp chí XYZ

Tài liệu tham khảo

Tiếng Việt

- [10] D.-H. Trường Đại học Khoa học Tự nhiên. “Quy định và hướng dẫn thực hiện luận văn thạc sĩ,” Accessed: Jun. 6, 2015. [Online]. Available: http://www.hcmus.edu.vn/index.php?option=com_content&task=blogcategory&id=142&Itemid=506
- [13] D. Điền, *Xử lý ngôn ngữ tự nhiên*. Nhà xuất bản Đại học Quốc gia TP.HCM, 2006.
- [14] D. Q. Ban and H. V. Thung, *Ngữ pháp tiếng Việt*. Nhà xuất bản Giáo dục Việt Nam, 2006.

Tiếng Anh

- [1] Boehm, B. W. and Basili, V. R., “Software defect reduction top 10 list,” *Computer*, vol. 34, no. 1, pp. 135–137, 2001. DOI: 10.1109/2.962984 [Online]. Available: <https://doi.org/10.1109/2.962984>
- [2] Selenium. “Selenium browser automation project,” Accessed: Jun. 29, 2026. [Online]. Available: <https://www.selenium.dev/>
- [3] Microsoft. “Playwright documentation,” Accessed: Jun. 29, 2026. [Online]. Available: <https://playwright.dev/>

- [4] Research Nester. “Automation testing market size and growth forecasts 2026 to 2035,” Accessed: Jun. 29, 2026. [Online]. Available: <https://www.researchnester.com/reports/automation-testing-market/4740>
- [5] Capgemini, Sogeti, and OpenText. “World quality report 2025,” Accessed: Jun. 29, 2026. [Online]. Available: <https://www.sogeti.com/newsroom/world-quality-report-2025/>
- [6] Romano, A., Song, Z., Grandhi, S., Yang, W., and Wang, W., “An empirical analysis of UI-based flaky tests,” *arXiv preprint*, vol. arXiv:2103.02669 2021. [Online]. Available: <https://arxiv.org/abs/2103.02669>
- [7] Google AI for Developers. “Gemini API documentation,” Accessed: Jun. 29, 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs>
- [8] Browser Use Contributors. “Browser-use: Make websites accessible for AI agents,” Accessed: Jun. 27, 2026. [Online]. Available: <https://github.com/browser-use/browser-use>
- [9] Online. “Latex/source code listings,” Accessed: Jun. 6, 2015. [Online]. Available: http://en.wikibooks.org/wiki/LaTeX/Source_Code_Listings
- [11] Cavnar, W. B. and Trenkle, J. M., “N-gram-based text categorization,” in *In Proceedings of SDAIR-94, 3rd Annual Symposium on Document Analysis and Information Retrieval*, 1994, pp. 161–175.
- [12] Knuth, D. E., *The T_EXbook*. Addison-Wesley, 1984.
- [15] Zhang, K. and Shasha, D., “Simple fast algorithms for the editing distance between trees and related problems,” *SIAM Journal on Computing*, Volume 18 Issue 6, pp. 1245–1262, 1989.
- [16] Online. “Latex/floats, figures and captions,” Accessed: Jun. 6, 2015. [Online]. Available: http://en.wikibooks.org/wiki/LaTeX/Floats,_Figures_and_Captions

- [17] Online. “Latex/tables,” Accessed: Jun. 6, 2015. [Online]. Available: <http://en.wikibooks.org/wiki/LaTeX/Tables>

Phụ lục A

Ngữ pháp tiếng Việt

Đây là phụ lục.

Phụ lục B

Ngữ pháp tiếng Nôm

Đây là phụ lục 2.